NoSQL Database Design – Lab Sheet 4

Concise solutions for all 15 questions, using HBase Shell and Cassandra CQL. Each answer has its own copyable code block.

1\. Install and Configure HBase – Student Table

```text
create 'Student', 'personal', 'academic'
put 'Student', '101', 'personal:name', 'Amit'
put 'Student', '101', 'personal:age', '20'
put 'Student', '101', 'academic:course', 'BCA'
put 'Student', '101', 'academic:marks', '85'
put 'Student', '102', 'personal:name', 'Riya'
put 'Student', '102', 'personal:age', '21'
put 'Student', '102', 'academic:course', 'BCA'
put 'Student', '102', 'academic:marks', '90'
put 'Student', '103', 'personal:name', 'Raj'
put 'Student', '103', 'academic:course', 'BBA'
put 'Student', '104', 'personal:name', 'Neha'
put 'Student', '104', 'academic:course', 'BCA'
put 'Student', '105', 'personal:name', 'Karan'
put 'Student', '105', 'academic:course', 'BBA'
scan 'Student'
```

2\. HBase CRUD Operations – Employee Table

```text
create 'Employee', 'details'
```

```
put 'Employee', 'E101', 'details:name', 'Amit'

put 'Employee', 'E101', 'details:salary', '30000'

get 'Employee', 'E101'

scan 'Employee'

put 'Employee', 'E101', 'details:salary', '35000'

delete 'Employee', 'E101', 'details:salary'

deleteall 'Employee', 'E101'
```

3\. HBase Event Logging System

```text
create 'Events', 'user', 'event'

put 'Events', 'EV101', 'user:name', 'Amit'

put 'Events', 'EV101', 'event:timestamp', '2026-10-29T10:00:00'

put 'Events', 'EV101', 'event:type', 'login'

put 'Events', 'EV101', 'event:status', 'success'

put 'Events', 'EV102', 'user:name', 'Riya'

put 'Events', 'EV102', 'event:type', 'file-upload'

put 'Events', 'EV102', 'event:status', 'success'

get 'Events', 'EV101'

get 'Events', 'EV102'

scan 'Events'
```

4\. HBase Content Management System

```text
create 'CMS', 'content', 'author', 'metadata'
```

put 'CMS', 'C101', 'content:title', 'MongoDB Guide'

put 'CMS', 'C101', 'content:body', 'Introduction to MongoDB'

put 'CMS', 'C101', 'author:name', 'Amit'

put 'CMS', 'C101', 'metadata:status', 'draft'

put 'CMS', 'C101', 'content:title', 'HBase Guide'

get 'CMS', 'C101'

delete 'CMS', 'C101', 'metadata:status'

deleteall 'CMS', 'C101'

```

### 5\. Blogging Platform Using HBase

```text

create 'Blogs', 'blog', 'author', 'comments'

put 'Blogs', 'B101', 'blog:title', 'Tech News'

put 'Blogs', 'B101', 'blog:category', 'Technology'

put 'Blogs', 'B101', 'author:name', 'Amit'

put 'Blogs', 'B101', 'comments:c1', 'Great blog'

put 'Blogs', 'B102', 'blog:title', 'Travel Guide'

put 'Blogs', 'B102', 'author:name', 'Riya'

put 'Blogs', 'B103', 'blog:title', 'Food Blog'

put 'Blogs', 'B103', 'author:name', 'Raj'

get 'Blogs', 'B101'

scan 'Blogs'

```

6\. HBase Column-Family Data Model

```text

```
create 'College', 'personal', 'academic'

put 'College', '101', 'personal:name', 'Amit'

put 'College', '101', 'personal:city', 'Delhi'

put 'College', '101', 'academic:course', 'BCA'

put 'College', '101', 'academic:marks', '85'

put 'College', '102', 'personal:name', 'Riya'

put 'College', '102', 'academic:course', 'BBA'

scan 'College'
```

Explanation: `personal` and `academic` are column families. Columns such as `personal:name` and `personal:city` are qualifiers within a family. Different rows can contain different qualifiers.

7\. HBase Website Page-View Counter

```text
create 'WebsiteCounter', 'views'

incr 'WebsiteCounter', 'home', 'views:count', 1

incr 'WebsiteCounter', 'home', 'views:count', 1

incr 'WebsiteCounter', 'about', 'views:count', 1

get 'WebsiteCounter', 'home'

get 'WebsiteCounter', 'about'

scan 'WebsiteCounter'
```

8\. HBase User Sessions with TTL

```text
create 'Sessions', {NAME => 'session', TTL => 3600}

put 'Sessions', 'U101', 'session:token', 'ABC123'
```

put 'Sessions', 'U101', 'session:status', 'active'

get 'Sessions', 'U101'

scan 'Sessions'

```

Explanation: `TTL => 3600` sets a one-hour time-to-live for cells in the `session` family. Expired data may not disappear immediately from physical storage.

### 9\. HBase Row-Key Design for Horizontal Scaling

```text

create 'Logs', 'data'

put 'Logs', '00_USER101_LOG1', 'data:event', 'login'

put 'Logs', '01_USER102_LOG2', 'data:event', 'logout'

put 'Logs', '02_USER103_LOG3', 'data:event', 'upload'

scan 'Logs'

```

Explanation: A row key should distribute records across regions instead of concentrating writes on one key range. A prefix or hash-based strategy can reduce hotspots. Actual distribution depends on region boundaries.

10\. HBase Regions and RegionServers

```text

create 'LargeTable',

 {NAME => 'data'},

 {SPLITS => ['300', '600', '900']}

put 'LargeTable', '100', 'data:value', 'A'

put 'LargeTable', '400', 'data:value', 'B'

put 'LargeTable', '700', 'data:value', 'C'

put 'LargeTable', '950', 'data:value', 'D'

```
scan 'LargeTable'

list_regions 'LargeTable'
```

Explanation: Split keys create multiple regions. RegionServers host and serve these regions, enabling data distribution and parallel processing. Region placement and availability depend on the running cluster configuration.

11\. Cassandra – CollegeDB and Students Table

```sql
CREATE KEYSPACE CollegeDB
WITH replication = {
  'class': 'SimpleStrategy',
  'replication_factor': 1
};
USE CollegeDB;
CREATE TABLE Students (
  student_id int PRIMARY KEY,
  name text,
  course text,
  marks int
);
INSERT INTO Students VALUES (101, 'Amit', 'BCA', 85);
INSERT INTO Students VALUES (102, 'Riya', 'BCA', 90);
INSERT INTO Students VALUES (103, 'Raj', 'BBA', 78);
INSERT INTO Students VALUES (104, 'Neha', 'BCA', 88);
INSERT INTO Students VALUES (105, 'Karan', 'BBA', 80);
SELECT * FROM Students;
```

```

Note: Replication factor `1` is suitable for a local single-node lab setup. Use a suitable higher factor for a multi-node cluster.

### 12\. Cassandra CRUD – Employees Table

```sql

USE CollegeDB;

CREATE TABLE Employees (

  emp_id int PRIMARY KEY,

  name text,

  department text,

  salary int

);

INSERT INTO Employees

VALUES (1, 'Amit', 'IT', 30000);

SELECT * FROM Employees;

UPDATE Employees

SET salary = 35000

WHERE emp_id = 1;

DELETE FROM Employees

WHERE emp_id = 1;

```

13\. Cassandra Event Logging System

```sql

CREATE TABLE Events (

  user_id int,

```
  event_time timestamp,
  event_id uuid,
  event_type text,
  status text,
  PRIMARY KEY ((user_id), event_time, event_id)
) WITH CLUSTERING ORDER BY (event_time DESC);
INSERT INTO Events
(user_id, event_time, event_id, event_type, status)
VALUES (101, '2026-10-29 10:00:00', uuid(), 'login', 'success');
INSERT INTO Events
(user_id, event_time, event_id, event_type, status)
VALUES (101, '2026-10-29 11:00:00', uuid(), 'logout', 'success');
SELECT * FROM Events
WHERE user_id = 101;
SELECT * FROM Events
WHERE user_id = 101
AND event_time >= '2026-10-29 09:00:00'
AND event_time < '2026-10-29 12:00:00';
```

14\. Cassandra Availability and Consistency

```sql
CREATE KEYSPACE TestDB
WITH replication = {
  'class': 'SimpleStrategy',
  'replication_factor': 3
```

};

USE TestDB;

CREATE TABLE Students (

 id int PRIMARY KEY,

 name text,

 marks int

);

INSERT INTO Students (id, name, marks)

VALUES (1, 'Amit', 85);

```

Run these commands in `cqlsh` on a cluster with three replicas:

```text

CONSISTENCY ONE;

SELECT * FROM Students;

CONSISTENCY QUORUM;

SELECT * FROM Students;

CONSISTENCY ALL;

SELECT * FROM Students;